

Smart Room for Patients: IoT Architecture

Marta Torres Sánchez, Patricia Lidia Sánchez Rodríguez, and Samrina
Sarkar Sammi

Télécom SudParis

September 2026

Contents

1	Introduction	2
2	State of the Art	3
2.1	Simple test representation	4
3	System Architecture (C4 Methodology)	4
3.1	Level 1: System Context	5
3.2	Level 2: Containers	5
3.3	Level 3: Components	7
3.4	Level 4: Code Level Structure	7
4	Infrastructure & Cloud Strategy	9
5	Prototype and Demonstration	9
5.1	Sleep-Aware Room Adaptation	9
5.2	Medical Emergency Scenario	10
6	Evaluation of the Results	10
6.1	Tests	10
6.2	Business Value and Economic Viability	11
6.3	Security, Privacy, and Safety Considerations	11
6.4	Future Work	11
7	Conclusion	12
A	Interaction with AI Tools	12
A.1	C4 Architecture Mapping	12
A.2	Prototype Development and Logic Simulation	12

Abstract

This document presents the architectural design and a state of the 5art comparison for a Smart Hospital IoT System. The proposed architecture integrates physiological sensors and environmental actuators through a centralized decision service. The primary goal of this design is to increase productivity, enhance comfort and safety, and reduce "alarm fatigue", which is a dangerous sensory overload caused by excessive false alarms that desensitizes clinical staff and compromises patient safety [1].

Through intelligent data filtering, the system ensures faster emergency responses while autonomously improving patient room conditions via a bidirectional IoT workflow. Finally, an interactive prototype validates this approach by demonstrating two practical scenarios: sleep-aware room adaptation and medical emergency management.

List of Acronyms

- **A/C:** Air Conditioning
- **AI:** Artificial Intelligence
- **BLE:** Bluetooth Low Energy
- **C4:** Context, Containers, Components, Code (Architecture framework)
- **ICT:** Information and Communication Technology
- **IoT:** Internet of Things
- **IP:** Internet Protocol
- **JSON:** JavaScript Object Notation
- **MQTT:** Message Queuing Telemetry Transport
- **RFID:** Radio Frequency Identification
- **SPR:** Smart Patient Room
- **WSN:** Wireless Sensor Network

1 Introduction

The proposed Smart Hospital Patient Monitoring & Room Automation System addresses critical inefficiencies in traditional healthcare facilities. Continuous patient monitoring in outdated hospitals often generates excessive false-positive alerts, causing alarm fatigue, a sensory overload that desensitizes hospital staff and compromises patient safety [1]. Additionally, the lack of automated environmental controls forces staff to manually manage room comfort, reducing productivity.

To overcome these challenges, the proposed system integrates physiological sensors and environmental actuators through a centralized decision service. By using rule-based decisions assisted by AI to trigger alerts only during genuine emergencies and autonomously

adapting room conditions to the patient’s state, this architecture eliminates alarm fatigue, enhances patient comfort and maximizes efficiency.

2 State of the Art

Recent literature highlights the growing integration of IoT and AI in healthcare, though usually in isolated domains. For instance, Sendelbach [1] focuses on the clinical problem of ”alarm fatigue”, demonstrating how traditional physiological monitors overwhelm staff with false positives. To address data processing, Kwon [2] explores the use of Artificial Intelligence specifically for predicting patient deterioration based on clinical data, without environmental integration.

Conversely, research on Smart Patient Rooms (SPRs) typically focuses entirely on environmental comfort rather than medical emergencies. Dawson [3] explores the use of smart displays and automated lighting to improve the patient experience, while Ali [4] proposes IoT-based climate control interfaces.

At the network layer, architectures commonly rely on Wireless Sensor Networks (WSNs) and lightweight protocols. Baker [5] validates the use of MQTT and edge gateways for continuous health telemetry, proving its efficiency for high-frequency data transmission.

While these existing approaches successfully apply AI to clinical data [2], automate room lighting [3], or optimize MQTT telemetry [5], they treat physiological monitoring and room automation as entirely separate systems.



Figure 1: Overview of a smart patient room equipped with integrated automation technologies, smart displays, and environmental control interfaces [3].

What Our Solution Does Better

The main differentiating idea of our project is not that each individual feature is completely novel on its own, but rather that we integrate them into one cohesive, bidirectional IoT workflow:

Sensing → Communication → Contextualization → Decision → Alert or physical actuation

Unlike existing solutions that treat physiological sensors and environmental actuators as separate domains, our architecture utilizes a centralized IoT Core Server equipped with a Decider Engine and an MQTT broker. This tightly couples physiological data processing with environmental actuation. By processing MQTT telemetry through a centralized rule-based logic, our system transcends passive observation. For instance, a sudden drop in a patient’s breathing rate not only routes a critical alert to the medical staff’s dashboard and triggers the physical buzzer/light in the staff room, but it can also autonomously adjust the room’s lighting to assist the incoming emergency response team. By clearly separating notification logic from physical alert actuation, and configuration databases from time-series telemetry, our architecture provides a robust, scalable, and fully integrated smart hospital environment.

2.1 Simple test representation

To demonstrate the feasibility of our architecture, we defined a working vertical slice focusing on a single, end-to-end use case: a critical heart rate alert. As illustrated in Figure 2, this slice covers the four essential stages of the IoT workflow. First, the physiological sensor detects an abnormal heart rate. Second, the ingestion gateway translates the local protocol and publishes the data payload via the MQTT broker. Third, the Decider Engine processes the contextualized information and determines that the threshold has been breached. Finally, the system executes a visible action by simultaneously activating a physical warning light and buzzer, and updating the clinical monitoring dashboard.

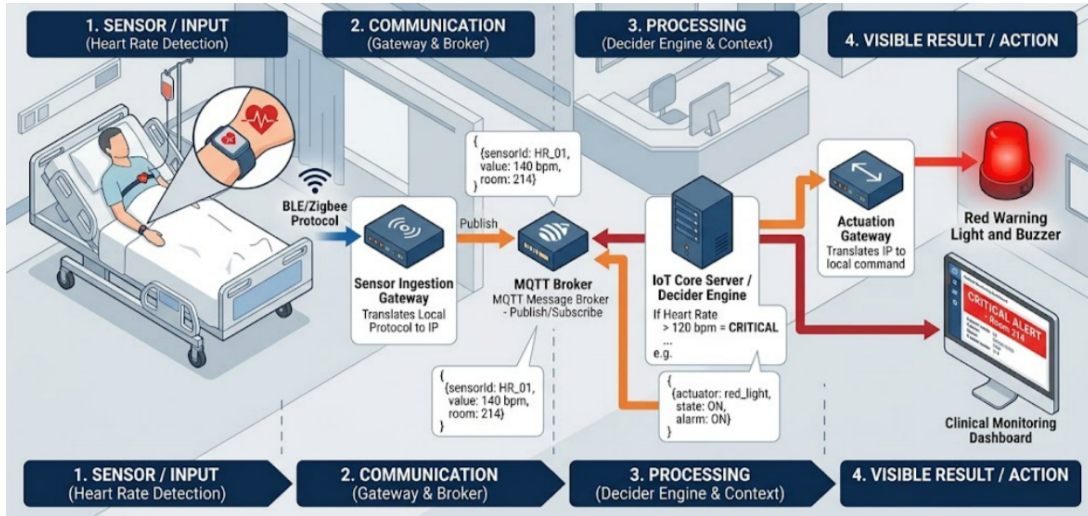


Figure 2: Working vertical slice demonstrating the end-to-end IoT workflow for a critical heart rate alert, from sensor data ingestion to physical and digital notification.

3 System Architecture (C4 Methodology)

To effectively describe the service and its major components, the system architecture is structured following the C4 methodology, which breaks down the design into Context, Containers, Components, and Code.



Figure 3: Summary of C4 Level 2 Container Diagram.

3.1 Level 1: System Context

At the highest level, the Smart Hospital System interacts with three main external entities: the patients (who generate physiological and environmental data), the medical staff (who receive alerts and monitor the dashboard), and the physical hospital environment (which is manipulated via actuators like A/C and blinds). The system’s primary goal is to act as the intelligent intermediary that automates the environment and ensures patient safety.

3.2 Level 2: Containers

The architecture follows a clear sensing and ingestion flow at the Container level, as shown in Figure 4 and summarized in Figure 3. To accommodate heterogeneous devices, physiological sensors (using BLE) and room sensors (using BLE or Zigbee) communicate with a Sensor Ingestion Gateway. This gateway translates local wireless protocols into IP-based MQTT messages using JSON payloads.

The Hospital IoT Message Broker acts as the communication backbone, routing these messages to the backend containers without making logical decisions. These independent containers manage the logic in the backend platform:

- **Data Context & Normalization Service:** Cleans incoming data and adds room/patient mappings.
- **Patient-in-Room Digital Twin:** Maintains a synchronized state of the physical environment and AI-derived context.
- **Patient & Room Decision & Control Service:** The core rule engine that takes final operational decisions.
- **Databases:** A Time-Series DB for historical telemetry and an Operational SQL DB for device mappings, alert states, preferences, and thresholds.

Additionally, the system uses different communication protocols: sensors use wireless links, backend services talk through lightweight MQTT messages, and staff dashboards connect using secure HTTPS and real-time WebSockets.

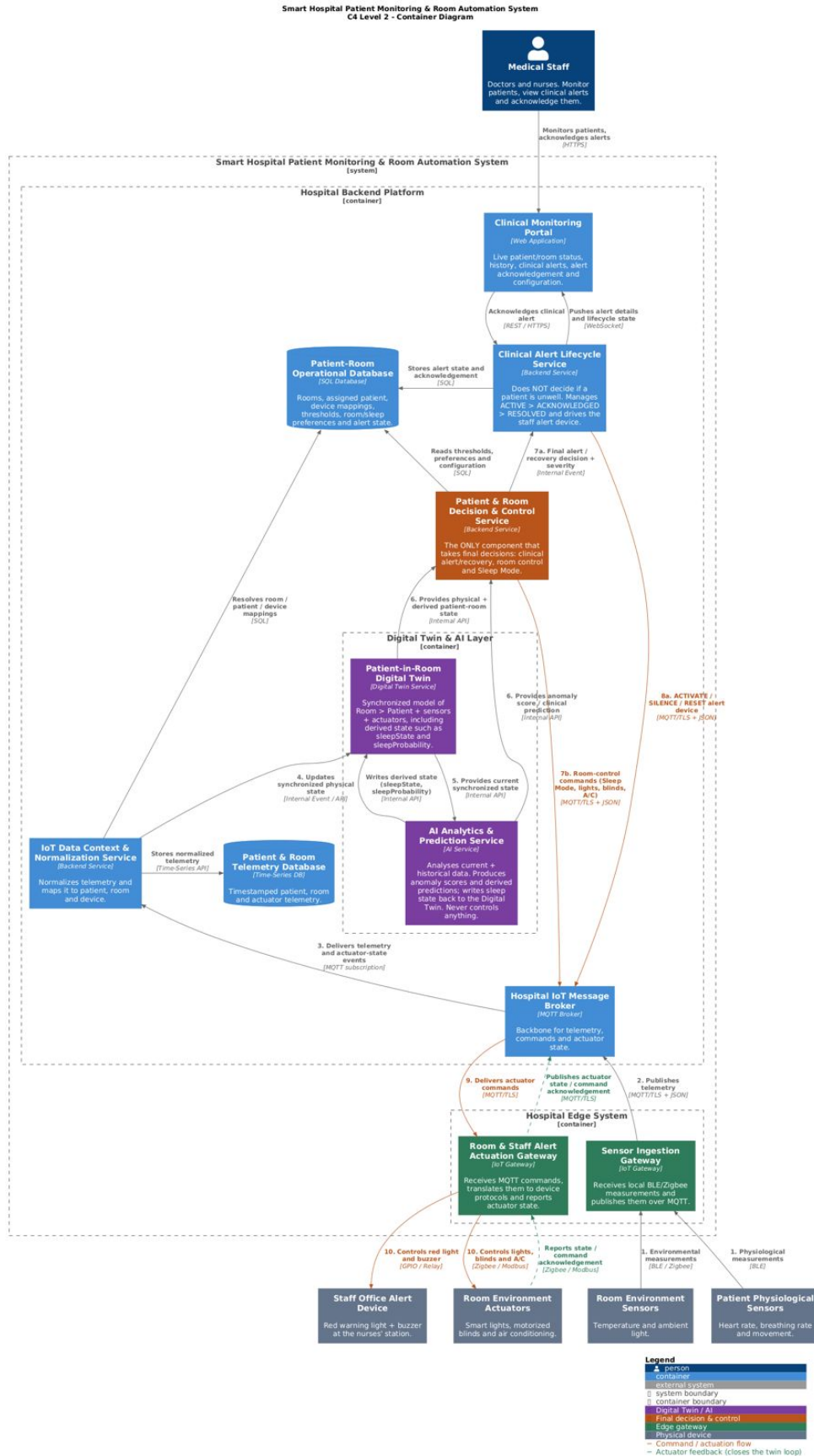


Figure 4: C4 Level 2 Container Diagram illustrating the overall architecture of the Smart Hospital system, detailing the data flow from physical sensors through the Edge gateways, IoT Message Broker, Digital Twin, AI Service, and the central Decision & Control Service.

3.3 Level 3: Components

Zooming into the Patient & Room Decision & Control Service, the architecture breaks down into specific internal components that handle the execution logic, as shown in Figure 5.

As the Digital Twin and AI services push updated states, the Digital Twin State Receiver and AI Result Receiver capture these payloads. A central Decision Coordinator bundles this information with the latest clinical thresholds fetched from the Configuration Repository.

This unified context is passed to the Clinical Alert Decision Engine and the Sleep Room Adaptation Engine, which apply strict rules. To avoid false positives caused by noisy sensor readings, a Sustained Condition Tracker ensures that alerts are only escalated if the anomaly persists across consecutive samples. Once a decision is made, the Clinical Alert Event Publisher or the Room Control Command Publisher triggers the respective workflow.

3.4 Level 4: Code Level Structure

At the Code level, the Decider Engine processes incoming MQTT messages. Instead of using complex internal rules, the system simply reads a JSON payload that combines the raw sensor data with the AI anomaly score. Figure 6 shows the structured telemetry message evaluated by the decision logic.

```
{
  "room_id": "ROOM_204",
  "patient_id": "PATIENT_P12",
  "timestamp": "2026-09-30T14:30:00Z",
  "physical_state": {
    "heart_rate_bpm": 135,
    "breathing_rate_rpm": 27,
    "ambient_light_lux": 280,
    "temperature_celsius": 24.5
  },
  "ai_derived_context": {
    "anomaly_score": 0.85,
    "sleep_state": "AWAKE"
  }
}
```

Figure 6: JSON payload structure enriched with AI context, received by the Decision Service.

The core execution logic parses these JSON fields and evaluates strict conditional thresholds to trigger automated actions via standard MQTT publish commands. The implementation branches into two primary operational workflows:

- **Clinical Alert Logic:** If the parsed `anomaly_score` exceeds the critical threshold (e.g., ≥ 0.75) across 3 consecutive samples, the script immediately publishes an

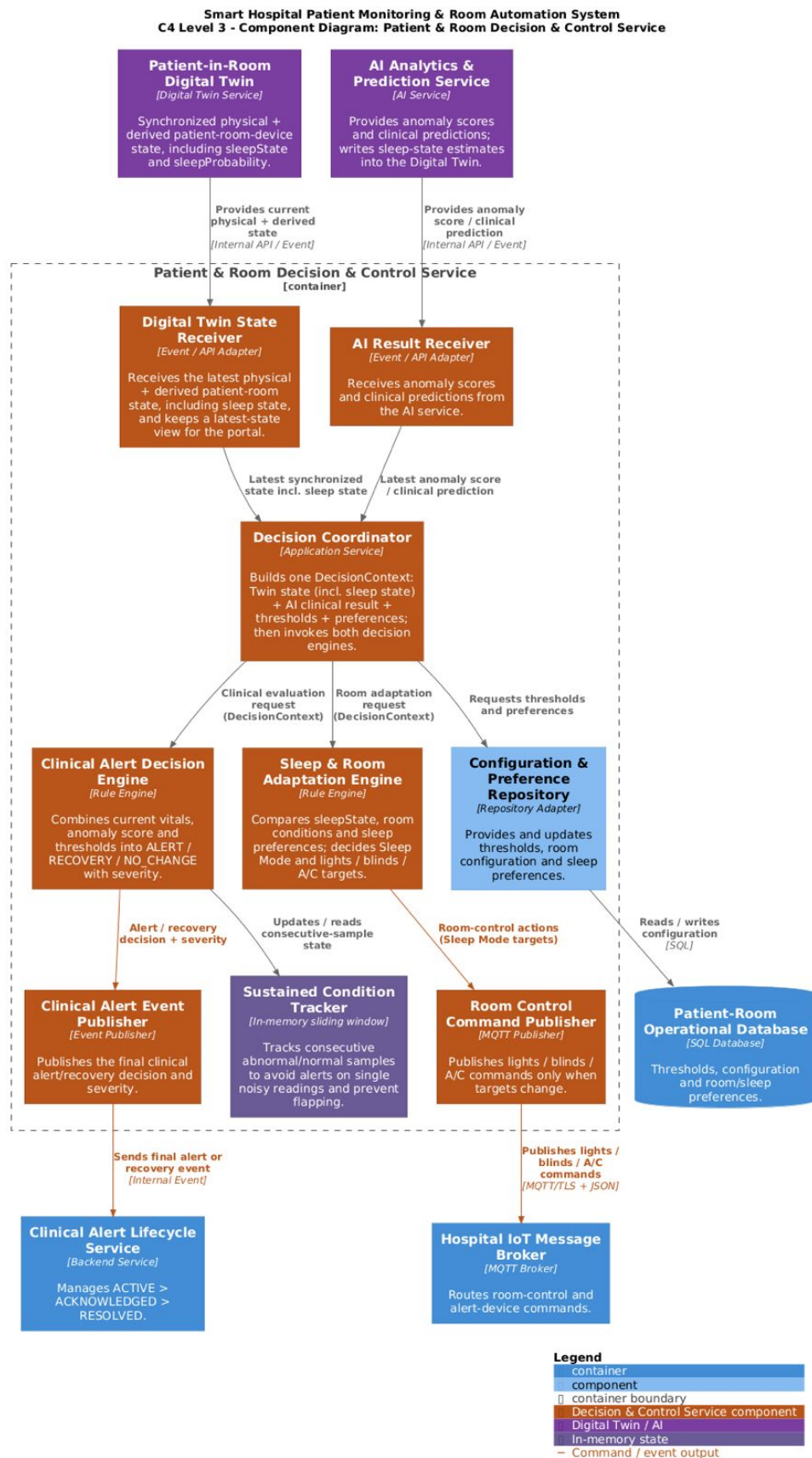


Figure 5: C4 Level 3 Component Diagram detailing the internal structure of the Patient & Room Decision & Control Service, showing how the Decision Coordinator combines the Digital Twin state and AI anomaly scores to trigger clinical alerts and room adaptation commands.

MQTT payload to the `staff_alarm` topic, triggering the physical buzzer and red warning light in the medical office.

- **Environmental Control Logic:** If the `sleep_state` indicates the patient is not `ASLEEP`, the script evaluates the physical room values against the configured preferences. If ambient light or temperature exceeds the sleep targets (> 60 lux or $> 22.5^{\circ}\text{C}$), it publishes an adjustment payload to the `room_control` topic to autonomously dim the lighting, close the motorized blinds, and activate the A/C.

4 Infrastructure & Cloud Strategy

To prioritize ultra-low latency, high availability, and strict data privacy, the system employs a Hybrid Edge-Cloud Architecture. Mission-critical components—such as the Sensor Gateways, MQTT Broker, Decider Engine, and Operational Database—run locally on the hospital’s edge network. This guarantees zero-latency execution for clinical alerts, even during external internet outages. Conversely, non-critical and resource-intensive tasks, such as long-term time-series storage and AI model retraining, are offloaded to the cloud.

Based on the high-frequency telemetry, each monitored patient publishes between 285 and 547 bytes of data every 2 simulated seconds. To manage storage, full-resolution data is retained for 7 days before being aggregated into 1-minute summaries.

5 Prototype and Demonstration

To validate the proposed architecture, an interactive prototype was developed in Python using Streamlit (available online at [<https://qkngzpnq3dfans5xpzr6rw.streamlit.app/>] (<https://qkngzpnq3dfans5xpzr6rw.streamlit.app/>)). The objective of the prototype is not to reproduce a complete hospital deployment, but to demonstrate two representative end-to-end vertical slices of the system and show how the main architectural components interact.

The prototype simulates the complete information flow from physical sensing to decision making and actuation. In both scenarios, sensor measurements are generated and associated with a specific patient and hospital room. These measurements are then used to update the Patient-in-Room Digital Twin, which maintains the current synchronized state of the patient, room, and relevant devices. The AI Analytics and Prediction Service processes this state and produces derived information, while the Patient & Room Decision and Control Service remains responsible for the final operational decision.

The demo therefore reflects the following common processing chain:

Sensors $\rightarrow$ Contextualization $\rightarrow$ Digital Twin $\rightarrow$ AI $\rightarrow$ Decision Service

From this common core, the system follows different paths depending on the type of situation detected. Two scenarios were implemented: sleep-aware room adaptation and a simulated medical emergency.

5.1 Sleep-Aware Room Adaptation

The first scenario demonstrates environmental automation. The patient starts in an awake state with excessive ambient light and a room temperature above the configured sleep

preference. Simulated physiological and environmental measurements are contextualized and synchronized with the Patient-in-Room Digital Twin.

The AI Analytics and Prediction Service estimates one of three demonstration states: AWAKE, DROWSY, or ASLEEP. The Decision Service combines this result with room conditions and configured preferences. When the room is too bright or too warm, it publishes commands through MQTT to close the blinds, reduce lighting, and activate the air conditioning.

The Digital Twin receives actuator feedback. As the simulated room conditions improve, movement decreases and the sleep probability increases, resulting in the transition from AWAKE to DROWSY and finally ASLEEP.

5.2 Medical Emergency Scenario

The second scenario demonstrates the clinical alert path. The patient’s simulated heart rate and breathing rate progressively increase. The measurements pass through the same sensing, contextualization, and Digital Twin pipeline used in the sleep scenario.

The AI Analytics and Prediction Service produces a simulated anomaly score. The Decision Service evaluates this score together with the Digital Twin state and configured thresholds. When the demonstration thresholds are exceeded for 3 consecutive samples, a high-severity clinical alert is generated.

The Clinical Alert Lifecycle Service manages three states: **ACTIVE** → **ACKNOWLEDGED** → **RESOLVED**. When **ACTIVE**, the warning light and buzzer are switched on. After staff acknowledgement, the buzzer is silenced while the visual warning remains active. When the simulated values drop to safe levels ($HR \leq 100$, $RR \leq 20$, anomaly score ≤ 0.25) for 3 consecutive samples, the alert becomes **RESOLVED** and both warning devices are switched off.

6 Evaluation of the Results

The two vertical slices demonstrate the main interactions of the proposed architecture. The MQTT-based workflow supports both environmental automation and clinical alert management while keeping AI analysis separate from operational decision-making. The prototype also demonstrates the alert lifecycle: **ACTIVE** → **ACKNOWLEDGED** → **RESOLVED**. Staff acknowledgement silences the buzzer while the visual warning remains active until the simulated condition is resolved.

The main limitation is that all sensor measurements, AI outputs, and actuator responses are simulated. The prototype therefore does not evaluate real sensor noise, network latency, packet loss, clinical effectiveness, or emergency response time. A real deployment would additionally require security, regulatory compliance, fault tolerance, and clinical validation.

6.1 Tests

The core testable requirements of the system are:

1. The system must trigger a physical red light and acoustic buzzer within 2 seconds of the Decider Engine receiving an AI anomaly score ≥ 0.75 (caused by sustained abnormal physiological readings).

2. The acoustic buzzer must silence immediately when a medical staff member acknowledges the alert on the dashboard, while the red light remains active.
3. The system must automatically switch off the physical red light and transition the alert to RESOLVED when the patient’s physiological values return to normal and the AI anomaly score drops below 0.25.

6.2 Business Value and Economic Viability

The adoption of this IoT architecture addresses critical healthcare challenges, specifically staffing shortages and rising operational costs, through three main avenues:

- **Maximizing Staff Productivity:** Automating environmental controls and filtering false alarms with AI eliminates “alarm fatigue,” allowing clinical staff to focus strictly on genuine medical emergencies.
- **Long-Term ROI and Energy Savings:** The initial infrastructure investment (CapEx) is offset by operational savings (OpEx), driven by staff time savings and automated reductions in room lighting and climate control.
- **Improved Patient Outcomes:** Ensuring faster responses to real emergencies minimizes adverse events. This reduces recovery times, extended-stay costs, and institutional liability.

6.3 Security, Privacy, and Safety Considerations

Handling sensitive patient data requires strict safeguards, such as encrypted MQTT communication, authenticated devices, and secure databases. A real deployment must ensure complete data privacy and network segmentation to protect both the hospital and the patients.

To prevent unintended physical actions, the architecture strictly separates AI analysis from operational control. The AI service only generates predictions and anomaly scores, while the central Decision Service evaluates explicit rules before triggering any room actuators or clinical alerts.

Finally, the current prototype uses synthetic physiological thresholds for demonstration purposes. It is a proof of concept and must not be interpreted as a certified clinical decision-support system or medical device.

6.4 Future Work

Future work could extend the prototype in three main directions. First, real BLE/Zigbee sensors and physical actuators could replace the simulated devices, allowing evaluation under realistic latency, noise, and packet-loss conditions.

Second, the AI component could be evaluated using real or appropriately anonymized datasets and extended toward predictive rather than purely reactive analytics. Integration with healthcare standards such as HL7/FHIR could also support interoperability with hospital systems.

Finally, a real deployment would require fault-tolerant infrastructure, stronger security mechanisms, regulatory compliance, and evaluation with healthcare professionals.

7 Conclusion

This project presented a Smart Patient Room architecture integrating physiological monitoring, environmental sensing, AI-based analytics, Digital Twin synchronization, and deterministic decision-making through an MQTT-based IoT workflow. The prototype demonstrated two vertical slices: sleep-aware room adaptation and clinical emergency alert management. A key design principle is the separation between AI analysis and operational control, with the Decision Service remaining responsible for final actions. Although the prototype relies on simulated data and is not clinically validated, it demonstrates the feasibility of integrating environmental automation and clinical alert management within a common IoT architecture.

A Interaction with AI Tools

Artificial Intelligence tools, primarily Large Language Models, were utilized as engineering tools to assist in the design and implementation phases. Following the project guidelines, below is the log of the main interactions:

A.1 C4 Architecture Mapping

- **What we asked:** We asked the AI to help map our handwritten system concept (sensing, MQTT, Decider Engine) into the formal C4 methodology levels.
- **What we kept/changed:** We kept the structural breakdown (Context, Containers, Components, Code) but changed the names of the services to match our specific room-centric approach (e.g., explicitly naming the *Sensor Data Context Service*).
- **How we checked it:** We reviewed the C4 definitions from the course materials to ensure the AI's classification accurately reflected the container and component levels.

A.2 Prototype Development and Logic Simulation

- **What we asked:** We requested boilerplate Python code to simulate the MQTT parsing logic and Streamlit UI for our two vertical slices (sleep adaptation and medical emergency).
- **What we kept/changed:** We kept the Streamlit layout structure and the JSON parsing logic, but we modified the clinical thresholds and the alert lifecycle states (ACTIVE → ACKNOWLEDGED → RESOLVED) to match our predefined rules.
- **How we checked it:** We ran the code locally, tested the UI buttons, and verified that the simulated physical actuators (red light/buzzer) responded exactly as dictated by the Decider Engine rules.

References

- [1] C. S. Sendelbach. Alarm fatigue. *Nursing Clinics of North America*, 47(3):375–382, 2012.

- [2] Hyuktae Kwon, Sunhee An, Ho-Young Lee, Won Chul Cha, Sungwan Kim, Minwoo Cho, and Hyoun-Joong Kong. Review of smart hospital services in real healthcare environments. *Healthcare Informatics Research*, 28(1):3–15, 2022.
- [3] Joshua Dawson, K. Jens Phanich, and Jason Wiese. Reenvisioning patient education with smart hospital patient rooms. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 7(4):1–23, 2023.
- [4] Haneen Ali, Astin Cole, and Gabby Panos. Transforming patient hospital experience through smart technologies. In *International Conference on Human-Computer Interaction*, pages 203–215. Springer, 2020.
- [5] Stephanie B Baker, Wei Xiang, and Ian Atkinson. Internet of things for smart healthcare: Technologies, challenges, and opportunities. *IEEE Access*, 5:26521–26544, 2017.